OminiNote — Google Drive Sync Architecture

1. Data Hierarchy

```
Notebook
 └─ Super Section (optional nesting, like OneNote's
Section Groups)
     └─ Section
         └─ Canvas (infinite in one direction,
paginated in the other)
             └─ Page (unit of sync)
```

- A **Canvas** behaves like OneNote's infinite canvas, except instead of one unbounded surface, it's composed of discrete **Pages** that the user scrolls through and appends to.
- Each **Page** is a single JSON file containing its content: text blocks, images, PDFs, strokes, positioning data, etc.
- The **Page** is the atomic unit of sync — not the canvas, section, or notebook. This is the most important design decision in the whole system, because it determines the size and frequency of what gets synced.

2. Page Object — Required Metadata

Every page JSON must carry sync metadata alongside its content:

```
{
  "page_id": "uuid-v4",
  "canvas_id": "uuid-v4",
```

```
  "section_id": "uuid-v4",
  "notebook_id": "uuid-v4",
  "position_index": 3,
  "last_modified": "2026-07-07T14:32:01.123Z",
  "last_modified_device": "device-uuid",
  "dirty": true,
  "content_version": 17,
  "content": { ... }
}
```

Field	Purpose
`page_id`	Globally unique, permanent identifier for the page. Never reused, never reassigned.
`canvas_id` / `section_id` / `notebook_id`	Physical location of the page — lets the cloud (and any device) resolve exactly where a page belongs without needing to re-derive it from folder structure.
`last_modified`	Timestamp of the last **edit**, not the last time the page was opened.
`last_modified_device`	Which device made that edit — needed for conflict resolution and debugging.
`dirty`	`true` = has local edits not yet confirmed synced. `false` = local state matches last known-synced cloud state.
`content_version`	Monotonic counter, incremented on every edit. This

Field	Purpose
	matters more than you'd think — see §5.

Dirty flag rules

- Opening/viewing a page: **no change** to `dirty` or `last_modified`.
- First edit (keystroke, stroke, image drop, etc.) since last sync: `dirty → true`, `last_modified` updated immediately.
- Successful upload + cloud acknowledgment: `dirty → false`.
- If a sync attempt fails (offline, error), `dirty` stays `true` — this is your retry queue, for free.

3. Change Propagation

- Each edited page is sent as its **own independent change message** — not bundled into one big payload. If a user edits 3 pages before connectivity resumes, that's 3 separate change messages, not 1.
- Each message contains: `page_id`, `last_modified`, `content_version`, and the full page JSON (or a diff — see §6).
- On reconnect, a device doesn't need to "ask" the cloud anything upfront beyond: *"here is the last_modified timestamp I last successfully synced, for each page I have locally."* The cloud compares and returns anything newer.

4. Conflict Resolution — Last-Write-Wins by Timestamp

Your proposed rule: cloud compares its stored `last_modified` for a `page_id` against the incoming one; whichever is later wins.

This works, **with caveats** (see §7). The comparison logic:

```
if incoming.last_modified > cloud.last_modified:
    accept incoming, overwrite cloud copy, dirty =
false
elif incoming.last_modified < cloud.last_modified:
    reject incoming, tell device to pull cloud's
version instead
else:
    tie-break using content_version, then device_id
(deterministic, arbitrary but consistent)
```

5. Why you also want `content_version` (not just timestamp)

Pure timestamp-based LWW has two real weaknesses:

1. **Clock skew.** Phones and desktops don't always agree on the exact time, especially if a device has been offline for a while or has a slightly wrong system clock. A device with a clock 5 minutes fast will always "win," even if its edit was actually older in wall-clock reality.
2. **Same-millisecond edits.** Rare, but possible with automated content, or if two devices batch-sync at once.

A monotonic per-page `content_version` (incremented locally on every edit, and by the cloud on every accepted write) gives you a tie-breaker independent of clock trust. Practically:

- Compare `content_version` first.
- Fall back to `last_modified` only if versions somehow match (shouldn't happen, but defensive).

This is a small addition to what you already described, not a redesign.

6. Full page vs. diff sync

Right now the plan sends the **whole page JSON** on every change. That's simplest to implement and fine at your current scale (text + occasional image references). Two things worth deciding now rather than later:

- **Images/PDFs should not live inside the page JSON as base64 blobs.** Store them as separate files in Drive, referenced by ID/URL from the JSON. Otherwise a tiny text edit forces a full re-upload of every embedded image on that page, which will make sync slow and burn Drive API quota fast.
- **Diffing (CRDT/operational transform) is probably overkill for v1.** Whole-page JSON sync is fine as long as pages stay reasonably small (a "page," not the whole infinite canvas). Revisit diffing only if you see real-world pages growing large or conflicts becoming frequent with multiple simultaneous editors.

7. Where this architecture needs sharpening

Good parts:

- Page-level granularity (not canvas- or section-level) is the right call — minimizes conflict surface and sync payload size.
- Explicit `dirty` flag decoupled from viewing is correct and avoids needless writes.
- Separate messages per changed page avoids one giant sync blob and unnecessary conflicts.

Gaps to close before you build:

1. **Google Drive is not a real-time sync backend — it's a file store.** There's no push mechanism telling other devices "a file changed" the instant it happens. You'll be polling (via `changes.list` / `Files: watch` push notifications, which Drive does support) or relying on the app opening/foregrounding to trigger a check. Decide now whether you want Drive's native **Changes API** (efficient, gives you a delta since last page-token, works well for this use case) instead of manually scanning folders.
2. **One JSON file per page in Drive means one Drive file ID per page** — use that as your canonical `page_id` where possible, so you don't maintain a separate mapping table.
3. **You still need a manifest/index file** (per notebook or per section) listing all page IDs + their last known `last_modified` / `content_version`, so a device can figure out what's new without listing/opening every single page file on every sync. Otherwise sync cost scales with total pages instead of changed pages.
4. **Deletions and moves need explicit handling.** A missing page could mean "deleted" or "not yet synced" — you need a tombstone (a `deleted: true` flag with its own timestamp) rather than just removing the file, or one device will resurrect a page another device deleted.
5. **True LWW silently loses data on genuine conflicts** (same page edited offline on two devices between syncs). Acceptable for a single-user, multi-device notes app — just be aware it's "latest edit wins, older edit is gone," not "merge." If you ever want safety, keep the losing version as a timestamped conflict-copy instead of discarding it outright.

Bottom line: yes, this is a sound architecture for a single-user multi-device notes app on top of Google Drive — it's essentially what Drive's own sync model and most local-first note apps do (timestamp/version-based LWW at a fine-grained object level). The core additions worth locking in before implementation: a `content_version` counter alongside timestamp, a manifest file

per section/notebook to avoid full scans, tombstones for deletes, and images/PDFs stored outside the page JSON as referenced blobs.